HEX AWARE – JAVA TRACK

DAY 6 ASSIGNMENT

Functions, Aggregation, Grouping and Joins – MySQL 8.4 LTS

Prepared solution pack | 12 November 2026

Scope: This document addresses all three sets in the supplied assignment brief: Retail Sales Reporting, Insurance Policy Reporting, and Telecom Usage and Billing.

Schema note: The brief specifies business entities and query requirements, but does not provide table definitions or sample data. To make the program files executable, illustrative schemas and seed records are supplied. Adjust column names and sample values if your trainer has provided a separate database schema.

How to execute

For each folder, run schema.sql, then seed.sql, then reports.sql in MySQL 8.4 LTS. The queries use CURRENT_DATE() for date-sensitive conditions, so their results depend on the date they are executed.

Set 1 – Retail Sales Reporting

Eight reports: top products by revenue, monthly sales, customers with more than three orders, average basket value by city, unpaid orders, customers without orders, category revenue share, and highest-sales day. Also demonstrates NULL aggregates and WHERE versus HAVING.

Database schema

```
-- Retail Sales Reporting: schema
CREATE DATABASE IF NOT EXISTS day6_retail;
USE day6_retail;

CREATE TABLE customers (
  customer_id INT PRIMARY KEY, customer_name VARCHAR(100) NOT NULL,
  city VARCHAR(60), email VARCHAR(120), created_at DATE
);
CREATE TABLE products (
  product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL,
  category VARCHAR(60), unit_price DECIMAL(10,2) NOT NULL
);
CREATE TABLE orders (
  order_id INT PRIMARY KEY, customer_id INT NOT NULL, order_date DATE NOT NULL,
  payment_status VARCHAR(20), FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY, order_id INT NOT NULL, product_id INT NOT NULL,
  quantity INT NOT NULL, unit_price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Illustrative sample data

```
USE day6_retail;
INSERT INTO customers VALUES
(1,'Ananya Rao','Chennai','ananya@example.com','2025-01-10'),
(2,'Karthik Kumar','Bengaluru','karthik@example.com','2025-02-15'),
(3,'Meera Shah','Chennai','meera@example.com','2025-03-20'),
(4,'Arjun Nair','Kochi','arjun@example.com','2025-04-05'),
(5,'Divya Iyer','Madurai','divya@example.com','2025-05-12');
INSERT INTO products VALUES
(101,'Wireless Mouse','Accessories',799.00),
(102,'Mechanical Keyboard','Accessories',2499.00),
(103,'USB-C Hub','Electronics',1899.00),
(104,'Laptop Stand','Accessories',1299.00),
(105,'Webcam','Electronics',3299.00);
INSERT INTO orders VALUES
(1001,1,'2026-10-01','PAID'),(1002,1,'2026-10-05','PENDING'),
(1003,2,'2026-10-05','PAID'),(1004,2,'2026-10-12','PAID'),
(1005,3,'2026-10-12','UNPAID'),(1006,1,'2026-11-01','PAID'),
(1007,4,'2026-11-02','PAID'),(1008,2,'2026-11-02','PAID');
INSERT INTO order_items VALUES
(1,1001,101,2,799),(2,1001,102,1,2499),(3,1002,103,1,1899),
(4,1003,101,1,799),(5,1003,104,2,1299),(6,1004,105,1,3299),
(7,1005,103,2,1899),(8,1006,102,1,2499),(9,1006,101,1,799),
(10,1007,104,1,1299),(11,1008,105,2,3299);
```

Reporting queries

```
USE day6_retail;

-- 1. Top five products by revenue
SELECT p.product_id, p.product_name,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS revenue
FROM products p JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY revenue DESC LIMIT 5;

-- 2. Month-wise sales
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS sales_month,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS monthly_sales
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY sales_month;

-- 3. Customers with more than three orders (HAVING)
SELECT c.customer_id, UPPER(c.customer_name) AS customer_name,
       COUNT(o.order_id) AS order_count
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 3;

-- 4. Average basket value per city
SELECT c.city, ROUND(AVG(order_totals.order_value), 2) AS avg_basket_value
FROM customers c JOIN (
  SELECT o.order_id, o.customer_id, SUM(oi.quantity * oi.unit_price) AS order_value
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.order_id, o.customer_id
) order_totals ON order_totals.customer_id = c.customer_id
GROUP BY c.city ORDER BY avg_basket_value DESC;
```

```
-- 5. Orders without payment (LEFT JOIN)
SELECT o.order_id, CONCAT('ORDER-', o.order_id) AS order_reference,
       o.order_date, o.payment_status
FROM orders o LEFT JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.payment_status IS NULL OR o.payment_status <> 'PAID'
GROUP BY o.order_id, o.order_date, o.payment_status;

-- 6. Customers with no orders
SELECT c.customer_id, c.customer_name
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- 7. Product-category revenue share
SELECT p.category,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS category_revenue,
       ROUND(100 * SUM(oi.quantity * oi.unit_price) /
         (SELECT SUM(quantity * unit_price) FROM order_items), 2) AS revenue_share_percent
FROM products p JOIN order_items oi ON oi.product_id = p.product_id
GROUP BY p.category;

-- 8. Day with highest sales
SELECT o.order_date, ROUND(SUM(oi.quantity * oi.unit_price), 2) AS daily_sales
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_date ORDER BY daily_sales DESC LIMIT 1;

-- NULL demonstration using customers with and without orders.
-- COUNT(*) counts customer rows; COUNT(order_value), SUM and AVG ignore NULL order values.
SELECT COUNT(*) AS customer_rows,
       COUNT(t.order_value) AS customers_with_orders,
       SUM(t.order_value) AS sum_ignoring_nulls,
       AVG(t.order_value) AS avg_ignoring_nulls,
       SUM(COALESCE(t.order_value, 0)) AS sum_after_coalesce
FROM customers c
LEFT JOIN (
  SELECT o.customer_id, SUM(oi.quantity * oi.unit_price) AS order_value
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.customer_id
) t ON t.customer_id = c.customer_id;
-- WHERE filters rows before grouping; HAVING filters groups after aggregation.
```

Expected result-set guide

The eight reports return: product_id, product_name and revenue (up to five rows); sales_month and monthly_sales (one row per month); customer_id, uppercase customer_name and order_count (only counts above three); city and avg_basket_value; order_id, order_reference, order_date and payment_status for non-paid orders; customer_id and customer_name for customers with no orders; category, category_revenue and revenue_share_percent; and order_date plus daily_sales for the top day. The NULL demonstration returns customer_rows, customers_with_orders, sum_ignoring_nulls, avg_ignoring_nulls and sum_after_coalesce.

Set 2 – Insurance Policy Reporting

Eight reports: upcoming expiries, premium by product, customers with multiple active policies, claim ratios, policies without claims, average premium by age band, monthly new policies, and top agents. Includes a HAVING filter above 60% and INNER, LEFT and CROSS JOIN examples.

Database schema

```
-- Insurance Policy Reporting: schema
CREATE DATABASE IF NOT EXISTS day6_insurance;
USE day6_insurance;
CREATE TABLE customers (
 customer_id INT PRIMARY KEY, customer_name VARCHAR(100), date_of_birth DATE
);
CREATE TABLE agents (
 agent_id INT PRIMARY KEY, agent_name VARCHAR(100)
);
CREATE TABLE products (
 product_id INT PRIMARY KEY, product_name VARCHAR(100)
);
CREATE TABLE policies (
 policy_id INT PRIMARY KEY, policy_number VARCHAR(30), customer_id INT,
 product_id INT, agent_id INT, start_date DATE, end_date DATE,
 premium_amount DECIMAL(10,2), status VARCHAR(20),
 FOREIGN KEY(customer_id) REFERENCES customers(customer_id),
 FOREIGN KEY(product_id) REFERENCES products(product_id),
 FOREIGN KEY(agent_id) REFERENCES agents(agent_id)
);
CREATE TABLE claims (
 claim_id INT PRIMARY KEY, policy_id INT, claim_amount DECIMAL(10,2),
 claim_date DATE, FOREIGN KEY(policy_id) REFERENCES policies(policy_id)
);
```

Illustrative sample data

```
USE day6_insurance;
INSERT INTO customers VALUES
(1,'Asha Menon','1988-04-12'),(2,'Rahul Das','1975-08-21'),
(3,'Nila Joseph','1998-11-02'),(4,'Vikram Rao','1965-02-14');
INSERT INTO agents VALUES (1,'Priya Agent'),(2,'Sanjay Agent'),(3,'Farah Agent');
INSERT INTO products VALUES (1,'Health'),(2,'Life'),(3,'Motor');
INSERT INTO policies VALUES
(101,'POL-2026-001',1,1,1,'2025-12-01','2026-12-01',18000,'ACTIVE'),
(102,'POL-2026-002',1,2,1,'2026-01-10','2027-01-10',25000,'ACTIVE'),
(103,'POL-2026-003',2,1,2,'2026-02-01','2026-11-25',22000,'ACTIVE'),
(104,'POL-2026-004',3,3,3,'2026-03-15','2027-03-15',12000,'ACTIVE'),
(105,'POL-2026-005',4,2,2,'2024-06-01','2025-06-01',30000,'EXPIRED');
INSERT INTO claims VALUES
(1,101,9000,'2026-05-01'),(2,103,15000,'2026-07-10'),
(3,104,2000,'2026-08-15');
```

Reporting queries

```
USE day6_insurance;

-- 1. Policies expiring in the next 30 days (relative to execution date)
SELECT policy_id, CONCAT('***', SUBSTRING(policy_number, -4)) AS masked_policy,
       end_date, DATEDIFF(end_date, CURRENT_DATE()) AS days_to_expiry
FROM policies
WHERE end_date BETWEEN CURRENT_DATE() AND DATE_ADD(CURRENT_DATE(), INTERVAL 30 DAY)
  AND status = 'ACTIVE';

-- 2. Premium per product
SELECT pr.product_name, ROUND(SUM(p.premium_amount),2) AS total_premium
FROM products pr JOIN policies p ON p.product_id = pr.product_id
GROUP BY pr.product_id, pr.product_name;

-- 3. Customers with more than one active policy
SELECT c.customer_id, c.customer_name, COUNT(*) AS active_policy_count
FROM customers c JOIN policies p ON p.customer_id = c.customer_id
WHERE p.status = 'ACTIVE'
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(*) > 1;

-- 4. Claim ratio per product (claims / premiums).
-- Aggregate claims per policy first to avoid multiplying premium values.
SELECT pr.product_name,
       ROUND(100 * SUM(COALESCE(pc.claim_total,0)) / NULLIF(SUM(p.premium_amount),0),2) AS claim_ratio_percent
FROM products pr JOIN policies p ON p.product_id = pr.product_id
LEFT JOIN (SELECT policy_id, SUM(claim_amount) AS claim_total FROM claims GROUP BY policy_id) pc
 ON pc.policy_id = p.policy_id
GROUP BY pr.product_id, pr.product_name;

-- 5. Policies without claims
SELECT p.policy_id, p.policy_number
FROM policies p LEFT JOIN claims cl ON cl.policy_id = p.policy_id
WHERE cl.claim_id IS NULL;

-- 6. Average premium by age band (age at current date)
SELECT CASE
```

```
        WHEN TIMESTAMPDIFF(YEAR,c.date_of_birth,CURRENT_DATE()) < 30 THEN 'Under 30'
        WHEN TIMESTAMPDIFF(YEAR,c.date_of_birth,CURRENT_DATE()) < 45 THEN '30-44'
        WHEN TIMESTAMPDIFF(YEAR,c.date_of_birth,CURRENT_DATE()) < 60 THEN '45-59'
        ELSE '60+'
      END AS age_band,
      ROUND(AVG(p.premium_amount),2) AS average_premium
FROM customers c JOIN policies p ON p.customer_id = c.customer_id
GROUP BY age_band;

-- 7. Monthly new policies
SELECT DATE_FORMAT(start_date,'%Y-%m') AS policy_month, COUNT(*) AS new_policies
FROM policies GROUP BY DATE_FORMAT(start_date,'%Y-%m') ORDER BY policy_month;

-- 8. Top five agents by premium
SELECT a.agent_id, UPPER(a.agent_name) AS agent_name,
      ROUND(SUM(p.premium_amount),2) AS premium_sold
FROM agents a JOIN policies p ON p.agent_id = a.agent_id
GROUP BY a.agent_id, a.agent_name ORDER BY premium_sold DESC LIMIT 5;

-- Products with claim ratio above 60%, using GROUP BY and HAVING.
SELECT pr.product_name,
      ROUND(100 * SUM(COALESCE(pc.claim_total,0)) / NULLIF(SUM(p.premium_amount),0),2) AS ratio_percent
FROM products pr JOIN policies p ON p.product_id = pr.product_id
LEFT JOIN (SELECT policy_id, SUM(claim_amount) AS claim_total FROM claims GROUP BY policy_id) pc
 ON pc.policy_id = p.policy_id
GROUP BY pr.product_id, pr.product_name
HAVING SUM(COALESCE(pc.claim_total,0)) / NULLIF(SUM(p.premium_amount),0) > 0.60;

-- Join choices: INNER JOIN for matching policy/customer/product/agent rows;
-- LEFT JOIN to retain policies/products with no claims; CROSS JOIN for all
-- possible product x reporting-period combinations when a complete matrix is required.
-- CROSS JOIN example: every product paired with each reporting year.
SELECT pr.product_name, yr.report_year
FROM products pr CROSS JOIN (SELECT 2025 AS report_year UNION ALL SELECT 2026) yr;
```

Expected result-set guide

Expiry output is limited to active policies expiring from today through 30 days ahead. Premium and claim-ratio reports return one row per product. The multiple-policy report returns customers with more than one active policy. The no-claim report returns policies without a matching claim. Age bands and monthly policies are grouped outputs. The agent report returns up to five agents ordered by premium.

Set 3 – Telecom Usage and Billing

Eight reports: call minutes by plan, data-limit overruns, monthly billed revenue, overdue invoices, subscribers without usage, ARPU by circle, plan popularity, and top data consumers. Also demonstrates NULL handling and revenue by plan and circle.

Database schema

```
-- Telecom Usage and Billing: schema
CREATE DATABASE IF NOT EXISTS day6_telecom;
USE day6_telecom;
CREATE TABLE subscribers (
 subscriber_id INT PRIMARY KEY, subscriber_name VARCHAR(100), circle VARCHAR(60),
 data_limit_gb DECIMAL(8,2), plan_id INT
);
CREATE TABLE plans (
 plan_id INT PRIMARY KEY, plan_name VARCHAR(100), monthly_fee DECIMAL(10,2)
);
CREATE TABLE call_records (
 call_id INT PRIMARY KEY, subscriber_id INT, call_date DATETIME,
 duration_minutes DECIMAL(8,2), FOREIGN KEY(subscriber_id) REFERENCES subscribers(subscriber_id)
);
CREATE TABLE invoices (
 invoice_id INT PRIMARY KEY, subscriber_id INT, invoice_number VARCHAR(30),
 invoice_date DATE, due_date DATE, amount DECIMAL(10,2), paid_date DATE,
 FOREIGN KEY(subscriber_id) REFERENCES subscribers(subscriber_id)
);
CREATE TABLE data_usage (
 usage_id INT PRIMARY KEY, subscriber_id INT, usage_date DATE,
 data_used_gb DECIMAL(8,2), FOREIGN KEY(subscriber_id) REFERENCES subscribers(subscriber_id)
);
```

Illustrative sample data

```
USE day6_telecom;
INSERT INTO plans VALUES (1,'Basic',399),(2,'Plus',599),(3,'Unlimited',899);
INSERT INTO subscribers VALUES
(1,'Arun','Chennai',2.00,1),(2,'Bala','Coimbatore',5.00,2),
(3,'Charu','Chennai',10.00,3),(4,'Devi','Madurai',3.00,1);
INSERT INTO call_records VALUES
(1,1,'2026-10-01 10:00:00',12),(2,1,'2026-10-02 11:00:00',8),
(3,2,'2026-10-03 12:00:00',25),(4,3,'2026-10-05 09:00:00',40),
(5,2,'2026-11-01 14:00:00',15);
INSERT INTO invoices VALUES
(1,1,'INV-2026-001','2026-10-01','2026-10-16',399,'2026-10-10'),
(2,2,'INV-2026-002','2026-10-01','2026-10-16',599,NULL),
(3,3,'INV-2026-003','2026-10-01','2026-10-16',899,'2026-10-14'),
(4,4,'INV-2026-004','2026-11-01','2026-11-16',399,NULL);
INSERT INTO data_usage VALUES
(1,1,'2026-10-01',1.20),(2,1,'2026-10-02',1.10),
(3,2,'2026-10-01',3.50),(4,2,'2026-10-02',2.00),
(5,3,'2026-10-01',4.00);
```

Reporting queries

```
USE day6_telecom;

-- 1. Total call minutes per plan
SELECT p.plan_name, ROUND(SUM(cr.duration_minutes),2) AS total_call_minutes
FROM plans p JOIN subscribers s ON s.plan_id=p.plan_id
JOIN call_records cr ON cr.subscriber_id=s.subscriber_id
GROUP BY p.plan_id,p.plan_name;

-- 2. Subscribers exceeding their data limit
SELECT s.subscriber_id,s.subscriber_name,s.data_limit_gb,
       ROUND(SUM(du.data_used_gb),2) AS used_gb
FROM subscribers s JOIN data_usage du ON du.subscriber_id=s.subscriber_id
GROUP BY s.subscriber_id,s.subscriber_name,s.data_limit_gb
HAVING SUM(du.data_used_gb)>s.data_limit_gb;

-- 3. Month-wise revenue
SELECT DATE_FORMAT(invoice_date,'%Y-%m') AS revenue_month,
       ROUND(SUM(amount),2) AS billed_revenue
FROM invoices GROUP BY DATE_FORMAT(invoice_date,'%Y-%m') ORDER BY revenue_month;

-- 4. Invoices overdue by more than 15 days (unpaid)
SELECT invoice_id, CONCAT('INVOICE-',SUBSTRING(invoice_number,-3)) AS display_invoice,
       due_date, DATEDIFF(CURRENT_DATE(),due_date) AS days_overdue
FROM invoices
WHERE paid_date IS NULL AND DATEDIFF(CURRENT_DATE(),due_date)>15;

-- 5. Subscribers with no usage
SELECT s.subscriber_id,s.subscriber_name
FROM subscribers s LEFT JOIN data_usage du ON du.subscriber_id=s.subscriber_id
WHERE du.usage_id IS NULL;

-- 6. ARPU by circle (billed revenue / subscriber count)
SELECT s.circle, ROUND(SUM(i.amount)/COUNT(DISTINCT s.subscriber_id),2) AS arpu
FROM subscribers s LEFT JOIN invoices i ON i.subscriber_id=s.subscriber_id
GROUP BY s.circle;

-- 7. Plan popularity
```

```
SELECT p.plan_name, COUNT(s.subscriber_id) AS subscriber_count
FROM plans p LEFT JOIN subscribers s ON s.plan_id=p.plan_id
GROUP BY p.plan_id,p.plan_name ORDER BY subscriber_count DESC;

-- 8. Top ten data consumers
SELECT s.subscriber_id,s.subscriber_name,ROUND(SUM(du.data_used_gb),2) AS total_data_gb
FROM subscribers s JOIN data_usage du ON du.subscriber_id=s.subscriber_id
GROUP BY s.subscriber_id,s.subscriber_name ORDER BY total_data_gb DESC LIMIT 10;

-- NULL effect and correction: SUM/AVG ignore NULL; COALESCE converts NULL to zero.
SELECT s.subscriber_id, SUM(du.data_used_gb) AS raw_sum,
       AVG(du.data_used_gb) AS raw_average,
       COALESCE(SUM(du.data_used_gb),0) AS sum_with_zero,
       COALESCE(AVG(du.data_used_gb),0) AS average_with_zero
FROM subscribers s LEFT JOIN data_usage du ON du.subscriber_id=s.subscriber_id
GROUP BY s.subscriber_id;

-- Revenue per plan and circle using JOIN + GROUP BY + aggregate
SELECT p.plan_name,s.circle,ROUND(SUM(i.amount),2) AS revenue
FROM plans p JOIN subscribers s ON s.plan_id=p.plan_id
JOIN invoices i ON i.subscriber_id=s.subscriber_id
GROUP BY p.plan_id,p.plan_name,s.circle;

-- Note: COUNT(*) counts joined rows; COUNT(column) counts only non-NULL values.
```

Expected result-set guide

Call minutes and revenue are grouped by plan/month as requested. Data-limit output contains subscribers whose summed usage exceeds their limit. Overdue invoices are unpaid invoices more than 15 days past due. No-usage output contains subscribers without data_usage rows. ARPU is billed invoice amount divided by distinct subscriber count within a circle. Plan popularity includes plans with zero subscribers; top consumers returns up to ten subscribers.

Concept explanations and checks

WHERE vs HAVING

WHERE filters individual rows before grouping. HAVING filters grouped results after aggregate calculations. Example: WHERE status = 'ACTIVE' selects active policies before grouping; HAVING COUNT(*) > 1 keeps only customers whose active-policy group contains more than one row.

NULL behaviour

COUNT(*) counts rows. COUNT(column) ignores NULL values. SUM(column) and AVG(column) also ignore NULL values; when there are no non-NULL values, they return NULL. COALESCE(value, 0) or IFNULL(value, 0) can replace a NULL result with zero when zero is the intended business interpretation.

Join selection

INNER JOIN returns only matching records and is suitable when the report requires related entities. LEFT JOIN preserves every row from the left table even when no match exists, which is necessary for finding orders/customers/policies without related rows. CROSS JOIN returns every possible pair and is useful for a complete product-by-period matrix.

Aggregation correctness

When joining a one-to-many table such as claims to policies, aggregate the many-side first (for example, claims per policy) before summing premiums. This avoids multiplying premium values when a policy has multiple claim rows.

Expected output limitation

Exact numeric output is not fixed by the assignment brief because no official schema or dataset was supplied. Use the included seed scripts to reproduce outputs; date-based rows can vary as CURRENT_DATE() advances.